



哈爾濱工業大學  
HARBIN INSTITUTE OF TECHNOLOGY



# 第一讲 网络与分布式系统概述

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

## 讲在课程之前

- 为什么“人工智能”专业要有这门课？
- 从人工智能的国际竞争说起
- 从“计算机系统”到“协同的计算机系统”

# 这门课讲什么？

| 序号 | 内容         | 要求                                                   |
|----|------------|------------------------------------------------------|
| 1  | 网络与分布式系统概述 | 理解课程目标与分布式系统在AI中的重要性；掌握网络核心概念与分层模型；理解分布式系统的目标与挑战。    |
| 2  | 网络通信基础     | 掌握TCP可靠传输与拥塞控制机制；理解IP寻址与DNS工作原理；掌握Socket与RPC编程模型。    |
| 3  | 分布式系统核心    | 深入理解RPC的内部机制与实践问题；掌握分布式协调与领导者选举算法；掌握Raft共识算法的原理与流程。  |
| 4  | 分布式存储与计算   | 理解分布式存储系统架构与一致性模型；掌握MapReduce等分布式计算框架思想；了解云原生与容器化技术。 |

# 这门课讲什么？

| 序号 | 教学内容           | 教学要求                                                      |
|----|----------------|-----------------------------------------------------------|
| 5  | 分布式机器学习导论与数据并行 | 理解分布式机器学习的驱动力与挑战；掌握同步SGD与数据并行原理；深入理解并掌握 Ring-AllReduce算法。 |
| 6  | 模型与流水线并行       | 掌握模型并行与张量并行的切分策略；理解流水线并行的原理与气泡问题解决方案。                     |
| 7  | 分布式训练内存优化      | 掌握混合精度训练与梯度检查点技术；深入理解ZeRO优化器的核心思想与阶段。                     |
| 8  | 大模型推理与前沿专题     | 理解大模型推理的挑战与优化技术（如持续批处理）；了解分布式AI系统的前沿发展与案例。                |

# 这门课你将学到什么?

- 网络与分布式系统基本知识
- 从系统的角度看人工智能
- 从人工智能的角度看系统
- 如何面向一种模式固定的计算极致优化系统
- 如何设计一种系统友好的计算模式(人工智能模型)

## 学这门课要注意的

- “没有任何一门课程是需要先修课的”
- 不要惧怕机器学习，剥离模型语义，其计算模式是固定的
- 善用大模型
- 人工智能发展非常迅速，这门课内容可能很快就过时了，学习变中之不变
- 我们是在以这一代人工智能模型为例讲授下一代人工智能需要的核心思想！

# 让我们一起来学习!

James F. Kurose, Keith W. Ross. 《计算机网络：自顶向下方法》（原书第7版）. 机械工业出版社. 2018.

George Coulouris, Jean Dollimore, Tim Kindberg等. 《分布式系统：概念与设计》（原书第5版）. 机械工业出版社. 2018.

相关的论文，后续推荐

# 目录

## Contents

- 1 网络概述
- 2 协议实施的要素
- 3 分布式系统



# 目录

## Contents

1

**网络概述**

2

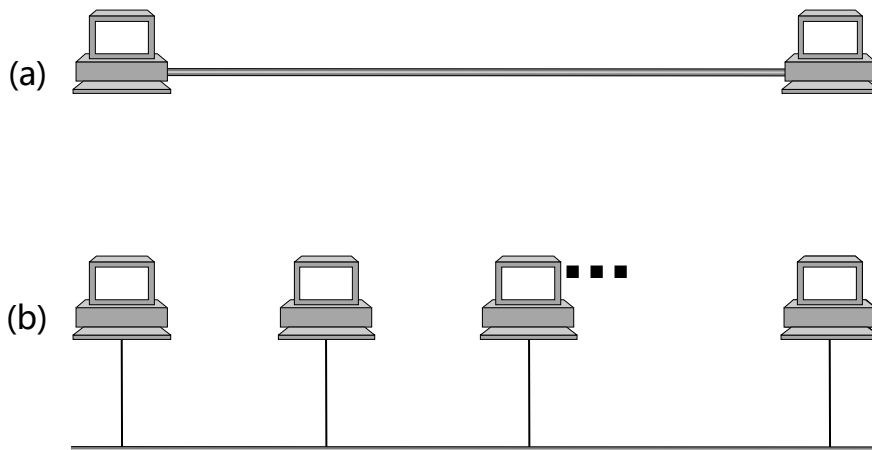
**协议实施的要素**

3

**分布式系统**

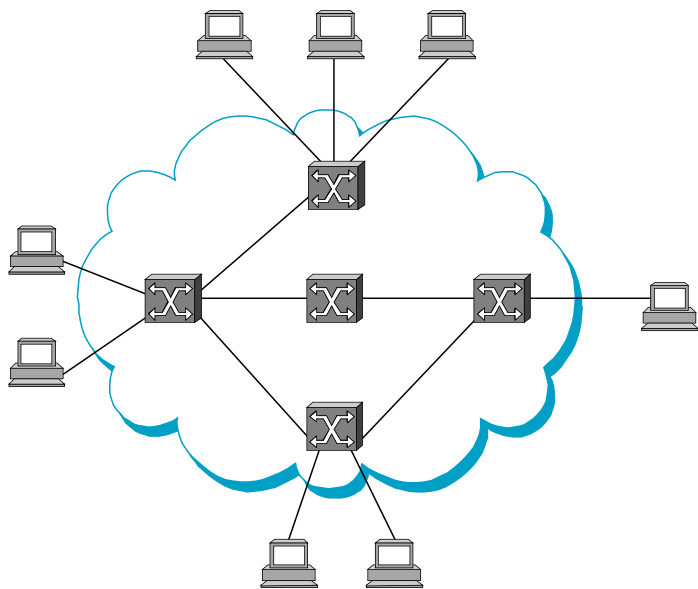
# 构成模块

- 节点:PC、专用硬件.....
- 主机
- 交换机
- 路由器
- 链接: 同轴电缆、光纤、双绞线、无线链接.....
- 点对点
- 多址

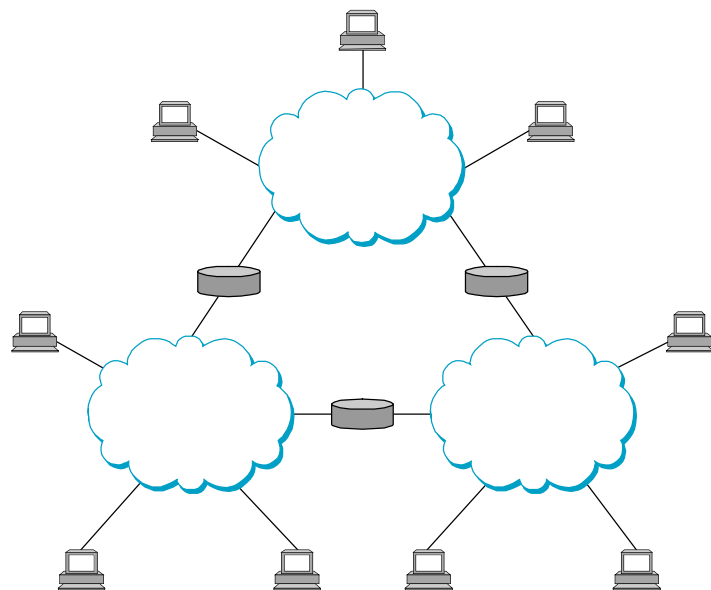


- 网络可以递归定义为.....

- 两个或多个节点通过链接连接,或



- 由一个节点连接的两个或多个网络



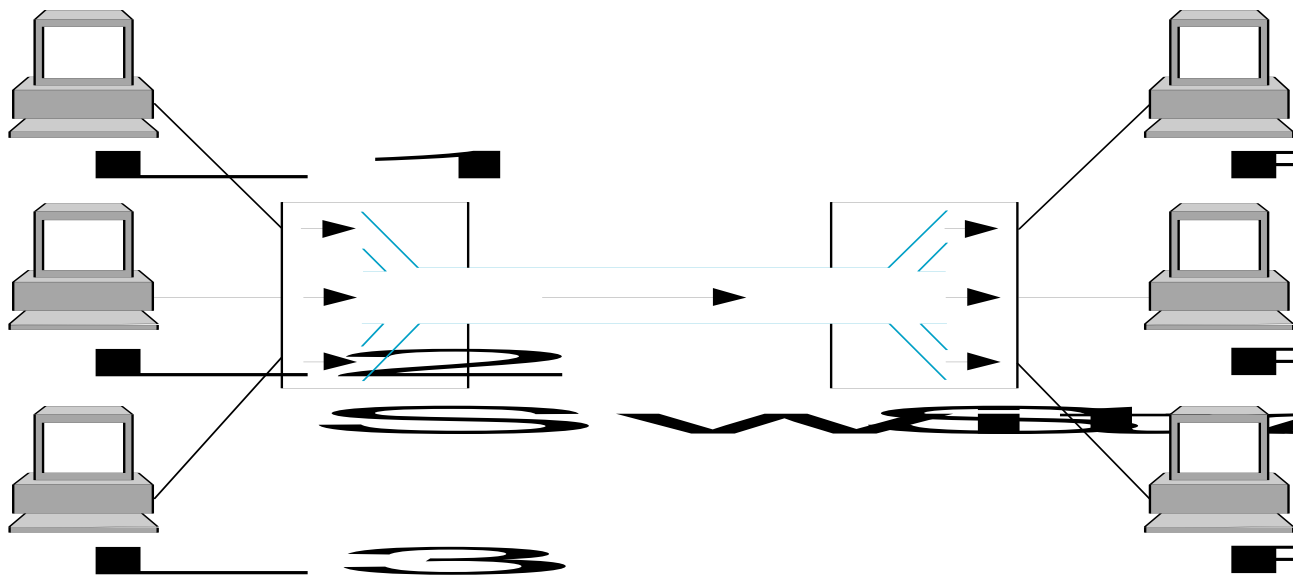
- 电路交换:携带比特流
  - 原电话网
- 数据包交换:存储和转发消息
  - 互联网

- 地址:标识节点的字节字符串
  - 通常独一无二
- 路由:根据其地址将消息转发到目标节点的过程
- 地址的种类
  - 单播:特定节点
  - 广播:网络上的所有节点
  - 多播:网络上节点的一些子集

# 多路复用

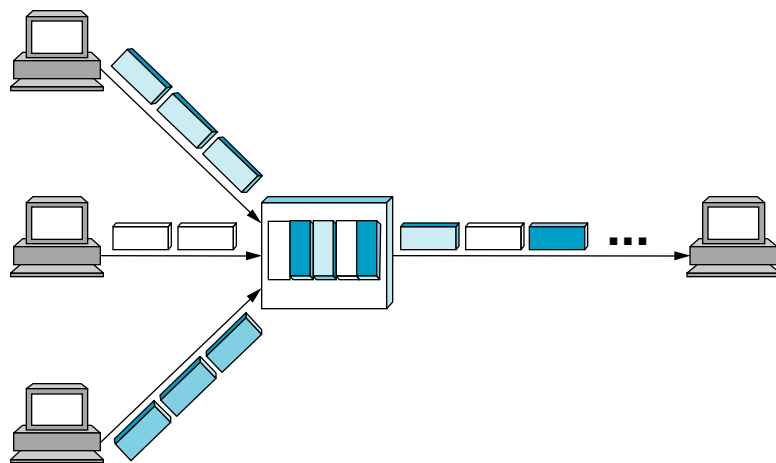
允许多个信号（或多个通信会话）在同一条物理线路上同时传输，而互不干扰

- 时分复用(TDM)：把时间划分为一段段等长的时隙，让多个信号轮流占用这些时隙。比如时隙1给A用，时隙2给B用，如此循环。
- 频分复用(FDM)：把物理信道的总带宽划分成若干个子频带（频道），每个子频带传输一路信号。



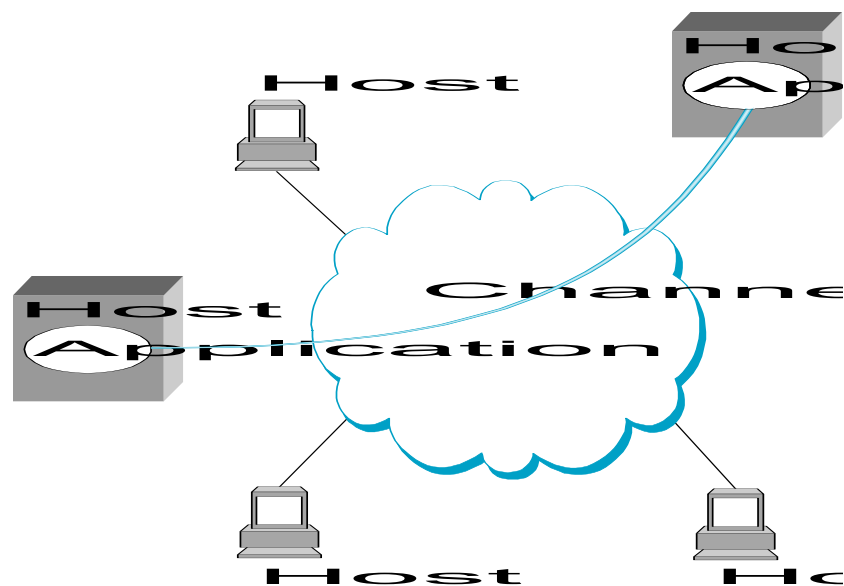
# 统计复用

- 按需分时
- 按数据包调度链接
- 来自不同来源的数据包在链路上交错
- 争夺链路的缓冲数据包
- 缓冲区(队列)溢出称为拥塞



# 进程间通信 (IPC)

- 将主机与主机的连接转变为进程与进程的通信。
- 填补应用程序期望与底层技术提供之间的空白。





- Request/Reply

- 分布式文件系统
- 数字图书馆(web)

- Stream-Based

- 视频:帧的顺序

- $1/4 \text{ NTSC} = 352 \times 240 \text{ pixels}$
- $(352 \times 240 \times 24) / 8 = 247.5 \text{ KB}$
- $30 \text{ fps} = 7500 \text{ KBps} = 60 \text{ Mbps}$

- 视频应用

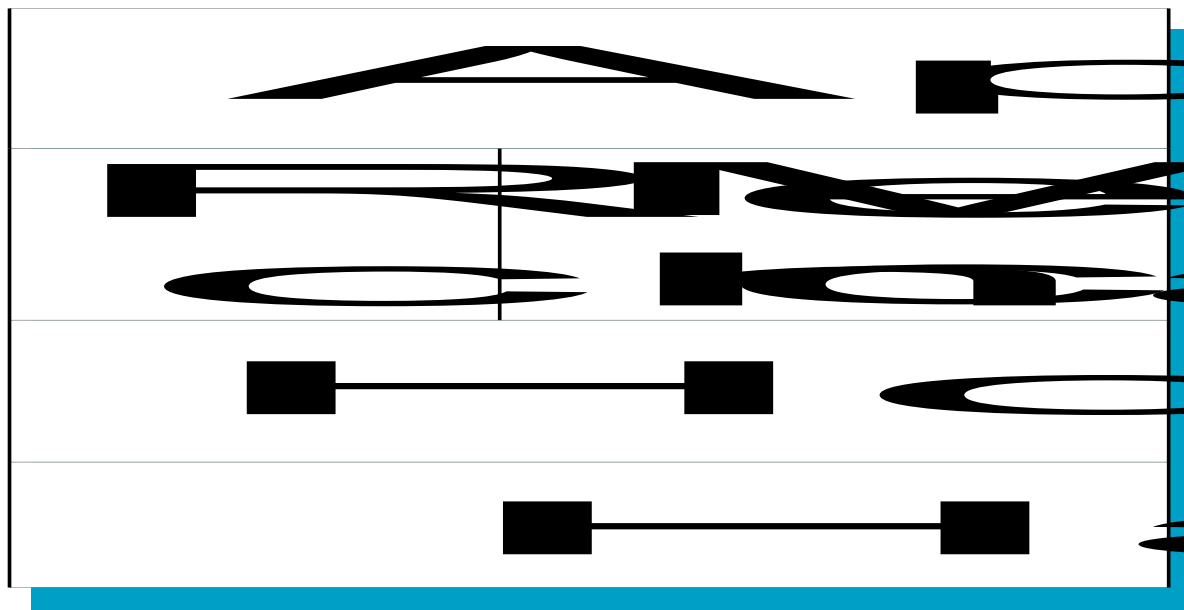
- 点播视频
- 视频会议

## 网络有什么问题？

- 位级错误(电气干扰)
- 数据包级错误(拥塞)
- 链路和节点故障
- 数据包被延迟
- 数据包不按顺序递送
- 第三者窃听

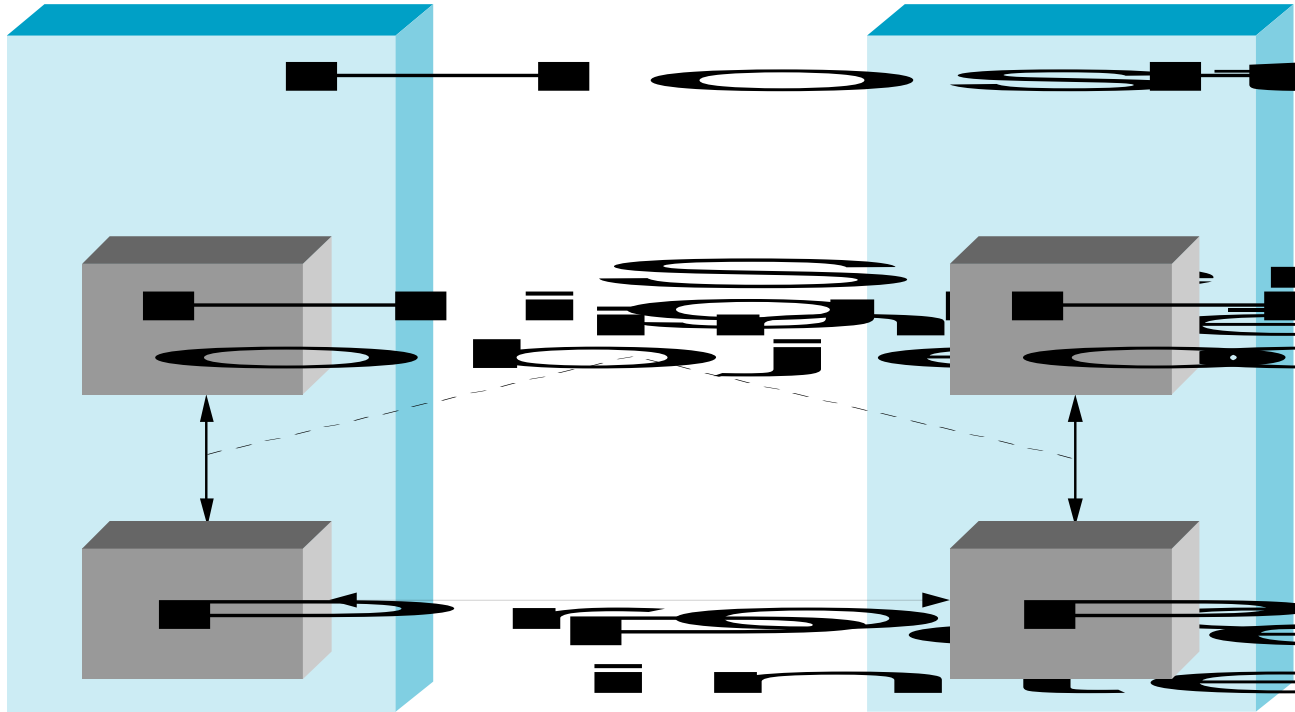
# 分层

- 用抽象来隐藏复杂性
- 抽象自然导致层次
- 每层的替代抽象



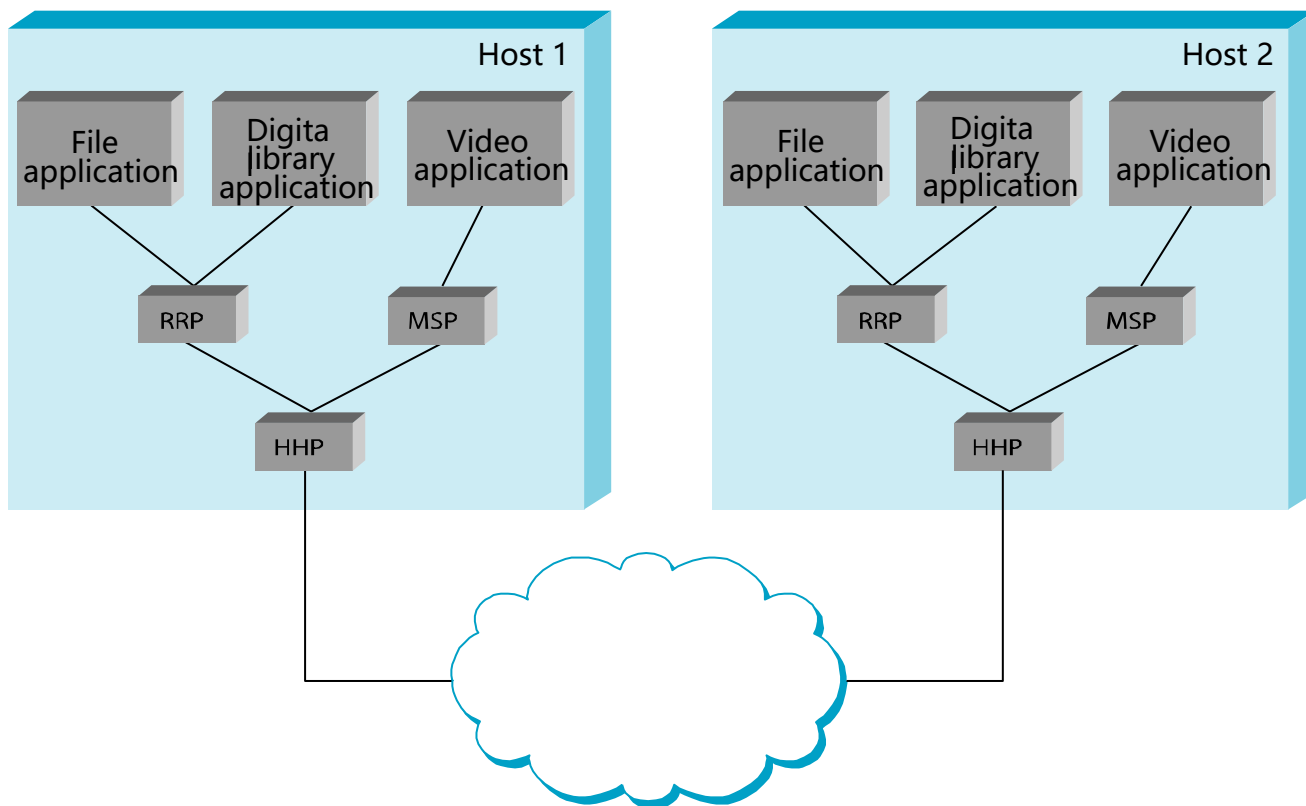
- 网络架构的构建块
- 每个协议对象有两个不同的接口
  - 服务接口:在此协议上的操作
  - 对等接口:与对等交换的消息
- "协议"一词过载
  - 点对点接口的规范
  - 实现此接口的模块

# 接口



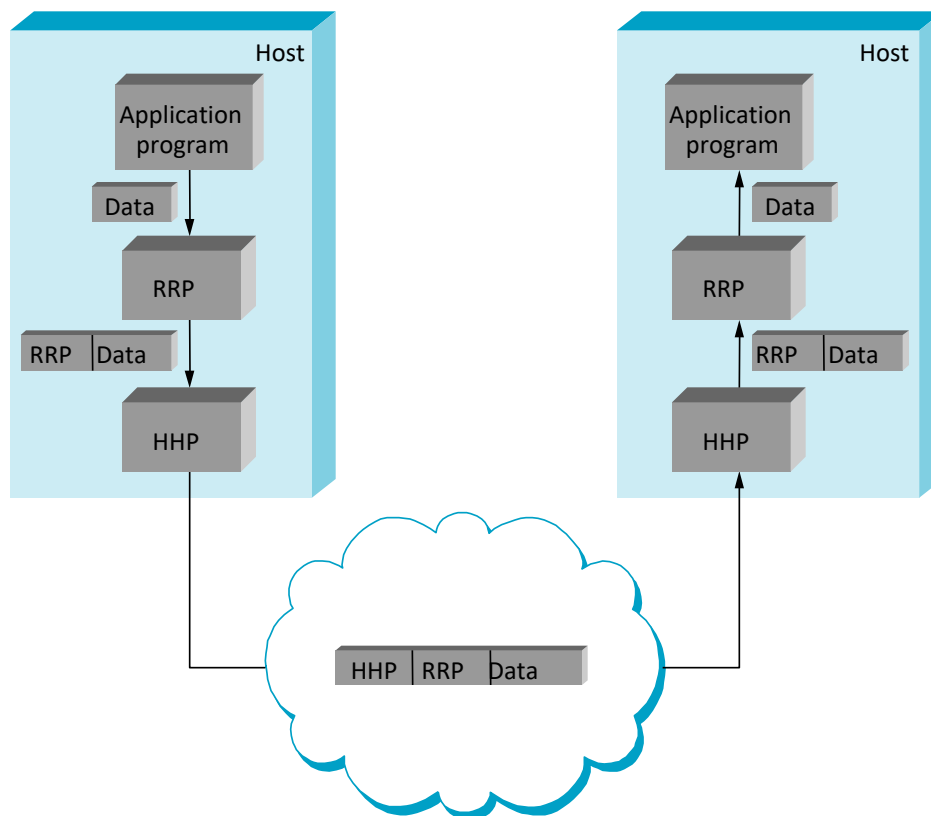
## 协议图

- 大多数点对点通信都是间接的
- 点对点仅在硬件层面直接



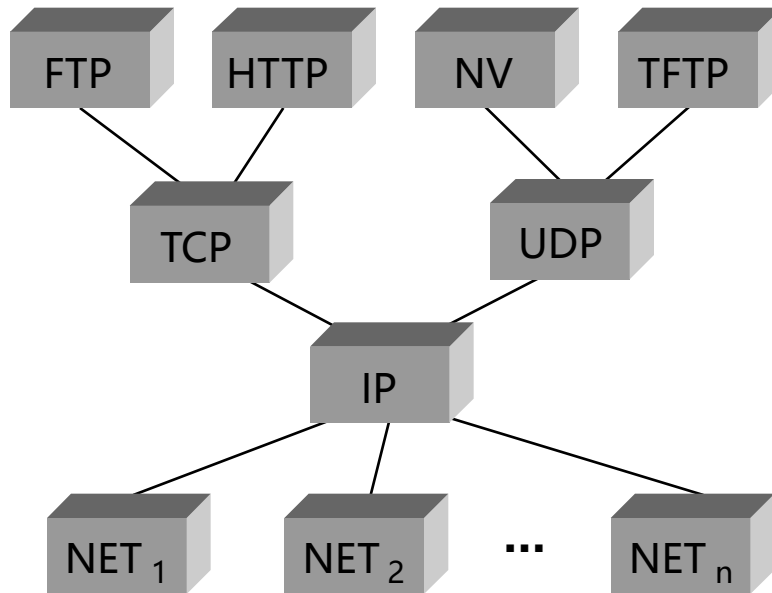
# 协议机制

- 多路复用和去多路复用(demux key)
- 封装(header/body)



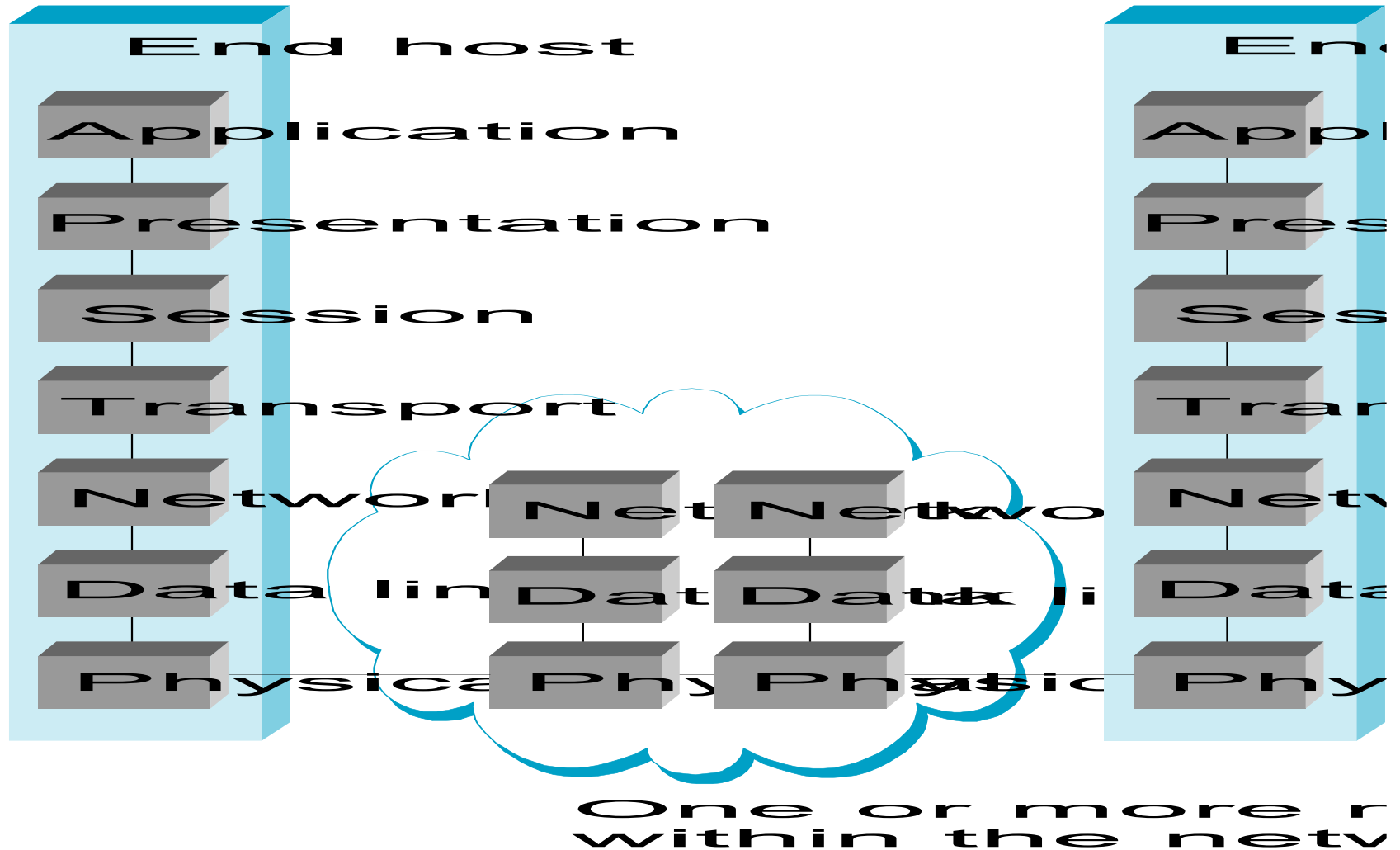
# 互联网架构

- 由 Internet Engineering Task Force (IETF) 定义
- 沙漏设计
- 应用程序与应用程序协议 (FTP, HTTP)





# ISO 架构



- 带宽(吞吐量)

- 以时为单位传输的数据
- 链接与端到端
- 单位

- KB =  $2^{10}$  bytes

- Mbps =  $10^6$  bits per second

- 延迟

- 从 A 点到 B 点发送消息的时间
- 单程与往返时间 (RTT)
- 组件

延迟 = 传播延迟 + 传输延迟 + 队列延迟

传播延迟 = 传播距离 / c

传输延迟 = 大小 / 带宽

## ● 相对重要性

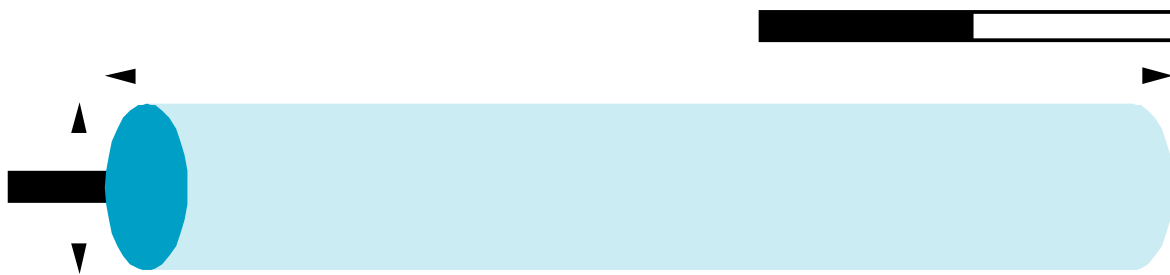
- 小数据量：传输时间可忽略，总时间主要由传播延迟决定，所以延迟的优化更重要。
  - 1-byte: 1ms vs 100ms 支配 1Mbps vs 100Mbps
- 大数据量：传输时间占绝大部分，总时间主要由带宽决定，所以带宽的优化更重要
  - 25MB: 1Mbps vs 100Mbps 支配 1ms vs 100ms

## ● 无限带宽

- 往返时间(RTT) 占主导地位
  - 吞吐量 =  $\text{TransferSize} / \text{TransferTime}$
  - $\text{TransferTime} = \text{RTT} + 1/\text{带宽} \times \text{TransferSize}$
- 1-MB文件转换为1-Gbps链路作为1-KB数据包到1-Mbps链路

## 延迟 x 带宽乘积

- "管道中"的数据量
- 通常相对于 RTT
- 例如:  $100\text{ms} \times 45\text{Mbps} = 560\text{KB}$



# 目录

## Contents

1

网络概述

2

协议实施的要素

3

分布式系统

- 创建一个socket

`int socket(int domain, int type, int protocol)`

- `domain = PF_INET, PF_UNIX`

- `type = SOCK_STREAM, SOCK_DGRAM, SOCK_RAW`

- 被动开放(在服务器上)

`int bind(int socket, struct sockaddr *addr, int  
addr_len)`

`int listen(int socket, int backlog)`

`int accept(int socket, struct sockaddr *addr, int  
addr_len)`

## Sockets (cont)

- 活动打开（在客户端上）

```
int connect(int socket, struct sockaddr *addr,  
            int addr_len)
```

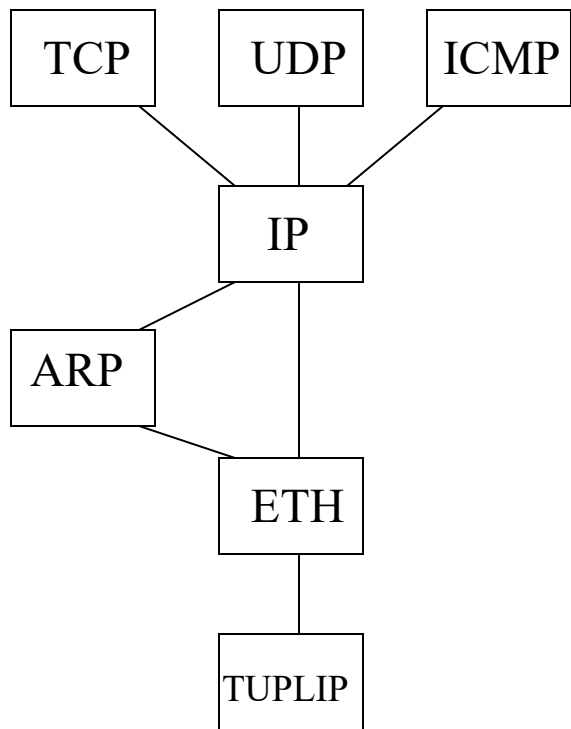
- 发送/接收消息

```
int send(int socket, char *msg, int mlen, int flags)  
int recv(int socket, char *buf, int blen, int flags)
```

- 配置多层
  - 静态与可扩展
- 过程模型
  - 避免上下文切换
- 缓冲模型
  - 避免数据副本



# 配置



**name=tulip;**  
**name=eth protocols=tulip;**  
**name=arp protocols=eth;**  
**name=ip protocols=eth,arp;**  
**name=icmp protocols=ip;**  
**name=udp protocols=ip;**  
**name=tcp protocols=ip;**

# Protocol Objects

- 主动打开  
**Sessn xOpen(Protl hlp, Protl llp,  
Part \*participants)**
- 被动打开  
**XkReturn xOpenEnable(Protl hlp,Protl llp,  
Part \*participant)**  
  
**XkReturn xOpenDone(Protl hlp,  
Protl llp,Sessn session,  
Part \*participants)**
- 去多路复用  
**XkReturn xDemux(Protl hlp, Sessn lls,  
Msg \*message)**

- 通道的本地端点
- 解释消息并保持通道状态
- 发送和接收消息的导出操作
- 业务

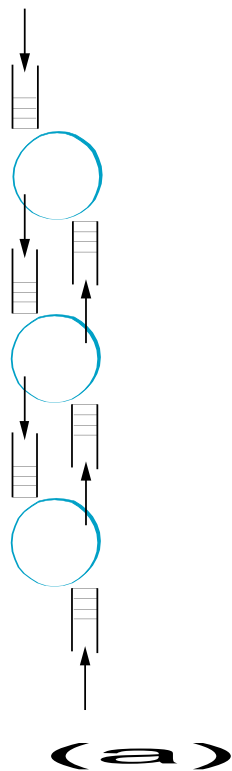
- 发送信息

**XkReturn xPush(Sessn lls,Msg \*message)**

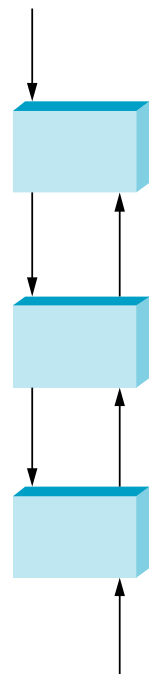
- 传递信息

**XkReturn xPop(Sessn hls, Sessn lls,  
Msg \*message,void \*hdr)**

# 过程模型

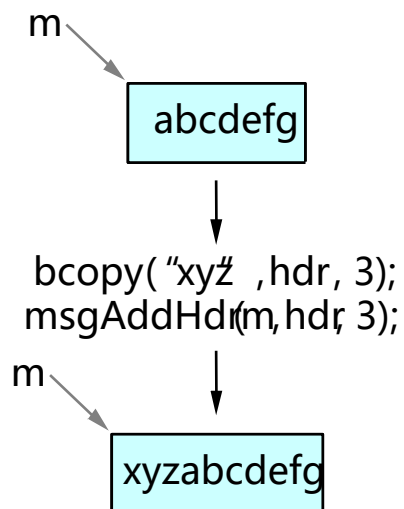


Process-per-  
Protocol

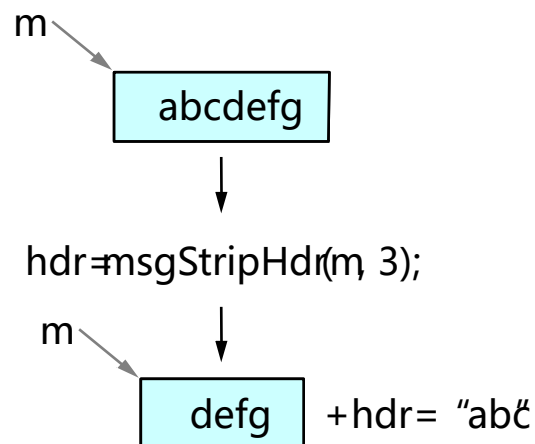


(b)  
Process-per-  
Message

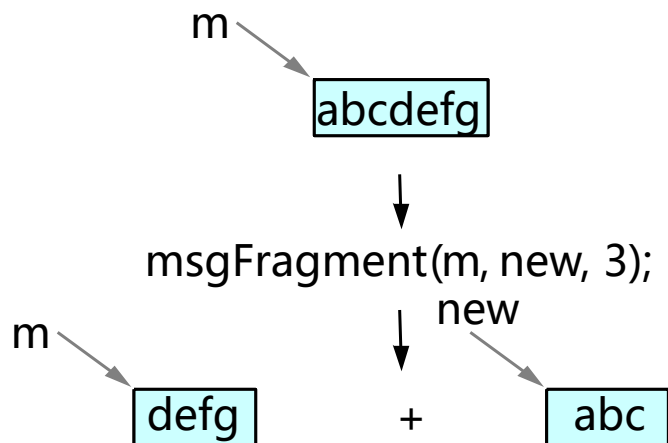
- 添加header



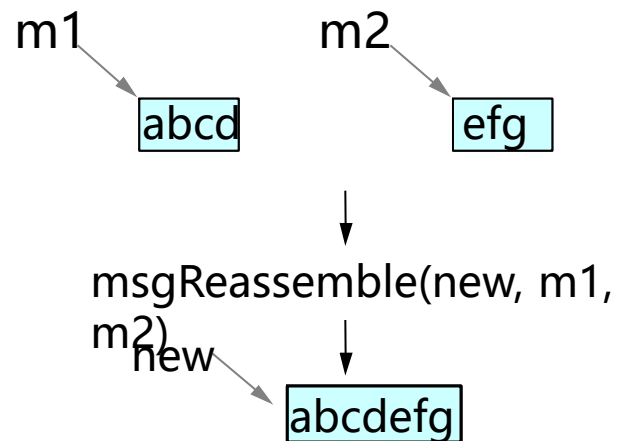
- 剥离header



- 消息分片



- 重新组合消息



- 超时调度&簿记活动
- 操作

**Event evSchedule(EvFunc function,  
void \*argument,  
int time)**

**EvCancelReturn evCancel(Event event)**

- 解复用数据包
- 操作

**Map mapCreate(int number,int size)**

**Binding mapBind(Map map,  
void \*key,  
void \*id)**

**XkReturn mapResolve(Map map,  
void \*key,void \*\*id)**



## 例子

```
static Sessn
aspOpen(Protl self, Protl hlp, Part *p)
{
    Sessn asp_s;
    Sessn lls;
    Activeld      key;
    ProtlState *pstate = (ProtlState *)self->state;

    bzero((char *)&key, sizeof(key));

    /* High level protocol must specify both */
    /* local and remote ASP port */
    key.localport = *((AspPort *) partPop(p[0]));
    key.remoteport = *((AspPort *) partPop(p[1]));
    /* Assume failure until proven otherwise */
    asp_s = XK_FAILURE;
```

```
/* Open session on low-level protocol */  
lls = xOpen(self, xGetDown(self, 0), p);  
if ( lls != XK_FAILURE )  
{  
    key.lls = lls;  
    /* Check for session in active map */  
    if (mapResolve(pstate->activemap, &key,  
        (void **)&asp_s) == XK_FAILURE)  
    {  
        /* Session not in map; initialize it */  
        asp_s = asp_init_sessn(self, hlp, &key);  
        if ( asp_s != XK_FAILURE )  
        {  
            /* A successful open */  
            return asp_s;  
        }  
    }  
}  
  
    /* Error has occurred */  
    xClose(lls);  
}  
return asp_s;
```

```
static XkReturn aspPush(Sessn s, Msg *msg)
{
    SessnState *sstate;
    AspHdr    hdr;
    void      *buf;

    /* create a header */
    sstate = (SessnState *) s->state;
    hdr = sstate->hdr;
    hdr.ulen = msgLen(msg) + HLEN;

    /* attach header and send */
    buf = msgAddrHdr(msg, HLEN);
    aspHdrStore(&hdr, buf, HLEN, msg);
    return xPush(xGetDown(s, 0), msg);
}
```

```
static XkReturn aspDemux(Protl self, Sessn lls, Msg *msg)
{
    AspHdr    h;
    Sessn     s;
    ActiveId  activeid;
    PassiveId passiveid;
    ProtlState *pstate;
    Enable    *e;
    void      *buf;

    pstate = (ProtlState *)self->state;

    /* extract the header from the message */
    buf = msgStripHdr(msg, HLEN);
    aspHdrLoad(&h, buf, HLEN, msg);

    /* construct a demux key from the header */
    bzero((char *)&activeid, sizeof(activeid));
    activeid.localport = h.dport;
    activeid.remoteport = h.sport;
    activeid.lls = lls;
```

```

/* see if demux key is in the active map */
if (mapResolve(pstate->activemap, &activeid,
    (void **)&s) == XK_FAILURE)
{
    /* didn't find an active session */
    /* so check passive map */
    passiveid = h.dport;
    if (mapResolve(pstate->passivemap, &passiveid,
        (void **)&e) == XK_FAILURE)
    {
        /* drop the message */
        return XK_SUCCESS;
    }

    /* port was enabled, so create a new */
    /* session and inform hlp */
    s = asp_init_sessn(self, e->hlp, &activeid);
    xOpenDone(e->hlp, s, self);
}
/* pop the message to the session */
return xPop(s, lls, msg, &h);
}

```

```
static XkReturn  
aspPop(Sessn s, Sessn ds, Msg *msg, void *inHdr)  
{  
    AspHdr *h = (AspHdr *) inHdr;  
  
    /* truncate message to length shown in header */  
    if ((h->ulen - HLEN) < msgLen(msg))  
        msgTruncate(msg, (int) h->ulen);  
  
    /* pass message up the protocol stack */  
    return xDemux(xGetUp(s), s, msg);  
}
```

# 目录

## Contents

- 1 网络概述
- 2 协议实施的要素
- 3 分布式系统**

- 我们为什么要开发分布式系统?
- 强大而廉价的微处理器(PC、工作站)的可用性,通信技术的不断进步

## ● 什么是分布式系统?

- 分布式系统是独立计算机的集合,在系统用户看来是一个系统。
- 例如:
  - 工作站网络
  - 分布式制造系统(例如自动化装配线)
  - 分支机构计算机网络



# 分布式系统优于集中式系统的优势

- 经济性:微处理器的集合比大型机具有更好的性价比。 低性价比:增加计算能力的经济方法。
- 速度:分布式系统的总计算能力可能比大型机多。 例如,10,000 个 CPU 芯片,每个芯片运行速度为 50 MIPS。 无法构建 500,000 MIPS 单处理器,因为它需要 0.002 纳秒的指令周期。 通过负载分配提高性能。
- 固有分布:有些应用程序是固有分布的。 例如,连锁超市。
- 可靠性:如果一台机器崩溃,整个系统仍然可以生存。 更高的可用性和更高的可靠性。
- 增量增长:计算能力可以小增量添加。 模块化可扩展性
- 另一个衍生力量:大量个人计算机的存在,需要人们协作和共享信息。

## 分布式系统优于独立 PC 的优势

- 数据共享:允许许多用户访问一个共同的数据库
- 资源共享:彩色打印机等昂贵的外围设备
- 沟通:加强人与人之间的沟通,例如电子邮件、聊天
- 灵活性:将工作量分散到可用的机器上

# 分布式系统的缺点

- 软件:难以为分布式系统开发软件
- 网络:饱和、有损传输
- 安全:易于访问也适用于秘密数据

- MIMD(多指令多数据)
- 紧耦合与松耦合
  - 紧耦合系统(多处理器)
    - 共享内存
    - 机间延迟短,数据速率高
  - 松耦合系统(多计算机)
    - 私有内存
    - 机间延迟长,数据速率低

## Bus与Switched MIMD

- Bus:连接所有机器的单一网络、背板、总线、电缆或其他介质。 例如,有线电视
- Switched:从机器到机器的单根电线,使用许多不同的布线模式。
- 多处理器(共享内存)
  - Bus
  - Switched
- 多计算机(私有内存)
  - Bus
  - Switched

- Bus-based multiprocessors
  - 缓存
  - 命中率
  - 缓存相干性
  - 直写缓存:立即传播写入
  - Snoopy Cache:监控其条目何时过时

- Switched Multiprocessors
  - 用于连接大量(例如超过 64 个)处理器
  - 横杆开关: $n^2$  开关点
  - omega network:  $2 \times 2$  开关, 用于  $n$  个 CPU 和  $n$  个存储器,  $\log n$  个开关级, 每个开关有  $n/2$  个开关,
  - 总  $(n \log n)/2$  开关
  - 延迟问题: 例如,  $n=1024$ , 从 CPU 到内存的 10 个切换级。 总共 20 个切换阶段。 100 MIPS 10 纳秒指令执行时间需要 0.5 纳秒的切换时间
  - NUMA (Non-Uniform Memory Access): 程序和数据的放置
  - 构建一个大型、紧密耦合的共享内存多处理器是可能的, 但困难且昂贵

- Bus-Based Multicomputers

- 易于建造
- 通信量小得多
- 速度相对较慢的 LAN(10-100 MIPS,而背板总线为 300 MIPS 以上)

- Switched Multicomputers

- 互连网络:例如,网格、超立方体
- 超立方:N维立方



- 软件对用户来说更重要
- 三种:
  1. 网络操作系统
  2. 分布式系统
  3. 多处理器分时

- 松耦合硬件上的松散耦合软件
- 通过局域网连接的工作站网络
- 每台机器都有高度的自主权
  - 登录机器
  - Rcp机器1：文件1机器2：文件2
- 文件服务器:客户端和服务端模型
- 客户端在文件服务器上挂载目录
- 最著名的网络操作系统:
  - Sun 的 NFS(网络文件服务器)用于共享文件系统 (Fig. 9-11)
- a few system-wide requirements: format and meaning of all the messages exchanged

- NFS架构

- Server exports 目录
- 客户端挂载导出的目录

- NFS协议

- 用于处理安装
- 对于读写:不打开/关闭,无状态

- NFS 实施

- 松耦合硬件上的紧密耦合软件
- 提供单系统映像或虚拟单处理器
- 单一的全局进程间通信机制、进程管理、文件系统;到处都是相同的系统调用接口
- 理想定义:

“分布式系统运行在一群计算机上,这些计算机没有共享内存,但在用户看来却是一台计算机.”

- 紧密耦合硬件上的紧密耦合软件
- 例如:高性能服务器
- 共享记忆
- 单次运行队列
- 传统文件系统与单处理器系统一样:中央块缓存

# 分布式系统的设计问题

- 透明度
- 灵活性
- 可靠性
- 性能
- 可伸缩性

# 1. 透明度

- 如何实现单系统图像,即如何使计算机的集合看起来像一台计算机。
- 对用户和应用程序隐藏所有分布,可以在两个层面上实现:
  - 1) 向用户隐藏分布
  - 2) 在较低的层次上,使系统对程序看起来透明。
  - 3) 1) 和 2) 需要统一的接口,例如访问文件、通信。

# 透明度的种类

- 位置透明:用户无法判断 CPU、打印机、文件、数据库等硬件和软件资源在何处。
- 迁移透明:资源必须可以自由地从一个地方转移到另一个地方,而不能改变其名称。  
例如,/usr/lee、/central/usr/lee
- 复制透明:操作系统可以在用户不注意的情况下额外复制文件和资源。
- 并发透明:用户不知道其他用户的存在。 需要允许多个用户同时访问同一资源。 锁定和解锁以相互排斥。
- 并行性透明性:自动使用并行性,无需显式编程。 分布式和并行系统设计者的圣杯。
- 用户并不总是想要完全透明:一台华丽的打印机在千里之外



## 2. 灵活性

使之易变

单体内核:系统调用被内核捕获并执行。

所有系统调用都由内核提供,例如 UNIX。

微内核:提供最少的服务。

- 1) IPC
- 2) 一些内存管理
- 3) 一些低级进程管理和调度
- 4) 低级 I/O

例如,Mach 可以支持多个文件系统、多个系统接口。

### 3. 可靠性

- 分布式系统应该比单一系统更可靠。 例如:3台机器,启动的概率为 .95。  $1-.05^{**3}$  上升的概率。
  - 可用性:系统可用时间的分数。 冗余改善了它。
  - 需要保持一致性
  - 需要安全
  - 容错:需要掩盖故障,从错误中恢复。

## 4. 性能

- 没有这一点,又何必为分布式系统而烦恼.
- 通信延迟导致的性能损失:
  - 细粒度并行: 高度交互
  - 粗粒度并行
- 由于使系统容错而导致的性能损失.

## 5. 可扩展性

- 系统与俱进,或过时。 在系统大小方面线性需要资源的技术是不可扩展的。 例如,基于广播的查询不适用于大型分布式系统。
- 瓶颈的例子
  - 集中式组件:单个邮件服务器
  - 集中表:单个 URL 地址簿
  - 集中式算法:基于完整信息的路由

- 计算机通过通信网络连接

- 广域网 (WAN)

- 连接遍布广阔地理区域的计算机

- 点对点或存储转发——数据通过一系列开关在计算机之间传输

- 交换机 -- 一种专用计算机,负责路由数据(以避免网络拥塞)

- 数据丢失的原因可能是:交换机崩溃、通信链路故障、交换机缓冲区有限、传输错误等。

i)电路切换——源与源之间的专用路径

目的地,例如电话连接.

浪费带宽(带宽 = 给定时间段内传输的数据量)

ii)分组交换 -- 信息或数据被分割成数据包

数据包独立路由, 更好地利用网络, 减少拆解和组装开销

ISO OSI参考模型

局域网(LAN)